Oldenburgische
Industrie- und Handelskammer

# Erklärung

## Bestätigung über die durchgeführte betriebliche Aufgabe[1]

(Diese Bestätigung ist als Deckblatt online einzureichen, gemeinsam mit dem Report/der Dokumentation.)

| **Prüfling (vollständige Anschrift und Telefonnummer)** | **Ausbildungsbetrieb (vollständige Anschrift)** |
|---|---|
| Vorname, Name | Firma |
| Straße, Hausnr. | Straße, Hausnr. |
| PLZ, Ort | PLZ, Ort |
| Tel.Nr.: | Tel.Nr.: |

Hinweis vorab: Aus Gründen der besseren Lesbarkeit wird auf die gleichzeitige Verwendung männlicher und weiblicher Sprachformen verzichtet. Sämtliche Personenbezeichnungen gelten gleichermaßen für alle Geschlechter.

Ausbildungsberuf

Bezeichnung der betrieblichen Aufgabe

**Erklärung des Prüflings**

Hiermit versichere ich, dass ich die betriebliche Aufgabe unter der Betreuung von

Verantwortlicher im Unternehmen

selbstständig durchgeführt und die Unterlagen selbstständig zusammengestellt habe.

Dokumente und Textpassagen, die ich nicht selbstständig erstellt habe, sind von mir gekennzeichnet.

Ort, Datum | Unterschrift des Prüflings

**Bestätigung des Ausbildungsbetriebes**

Wir bestätigen, dass die Angaben des Prüflings richtig sind.

Ort, Datum | Unterschrift des Verantwortlichen, der die Aufgabe betreut hat.

Ort, Datum | Unterschrift des Ausbilders

[1] Zur Vereinfachung wird einheitlich der Begriff „betriebliche Aufgabe“ verwendet. Gemeint sind die Fachaufgabe/die Projektarbeit/der betrieblicher Auftrag. Die unterschiedlichen Bezeichnungen entstehen durch die verschiedenen Berufe, die eine Aufgabe online einstellen.

Abschlussprüfung Sommer 2026

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

# Bereitstellung eines Microservices

## zur Synchronisation von dezentralisiert gespeicherten Kundendaten

Abgabetermin: Papenburg, den 08.05.2026

**Prüfungsbewerber:**
Niklas Hamburg
Kapitän-Rüter-Straße 81
26871 Papenburg

**Ausbildungsbetrieb:**
JOURNAWAY journawayGmbH
Am alten Handelshafen 1
26789 Leer

# Inhaltsverzeichnis

**Abbildungsverzeichnis** **III**

**Tabellenverzeichnis** **IV**

**Listings** **V**

**Abkürzungsverzeichnis** **VI**

**1 Einleitung** **1**
1.1 Projektumfeld . . . . . 1
1.2 Projektziel . . . . . 1
1.3 Projektbegründung . . . . . 1
1.4 Projektschnittstellen . . . . . 2
1.5 Projektabgrenzung . . . . . 2

**2 Projektplanung** **2**
2.1 Projektphasen . . . . . 3
2.2 Abweichungen vom Projektantrag . . . . . 3
2.3 Ressourcenplanung . . . . . 3
2.4 Entwicklungsprozess . . . . . 4

**3 Analysephase** **4**
3.1 Ist-Analyse . . . . . 4
3.2 Wirtschaftlichkeitsanalyse . . . . . 5
3.3 Anforderungsanalyse . . . . . 6
3.4 Nutzwertanalyse . . . . . 7

**4 Entwurfsphase** **8**
4.1 Zielplattform . . . . . 8
4.2 Architekturdesign . . . . . 8
4.3 Entwurf der Benutzeroberfläche . . . . . 8
4.4 Datenmodell . . . . . 9
4.5 Geschäftslogik . . . . . 9
4.6 Maßnahmen zur Qualitätssicherung . . . . . 10
4.7 Pflichtenheft/Datenverarbeitungskonzept . . . . . 10

**5 Implementierungsphase** **10**
5.1 Implementierung der Datenstrukturen . . . . . 10
5.2 Implementierung der Benutzeroberfläche . . . . . 10
5.3 Implementierung der Geschäftslogik . . . . . 11

**6 Abnahmephase** **11**

**7 Einführungsphase** **11**

**8 Dokumentation** **12**

**9 Fazit** **12**
9.1 Soll-/Ist-Vergleich . . . . . 12
9.2 Lessons Learned . . . . . 12
9.3 Ausblick . . . . . 13

**Literaturverzeichnis** **14**

**Eidesstattliche Erklärung** **15**

**A Anhang** **i**
A.1 Detaillierte Zeitplanung . . . . . i
A.2 Lastenheft (Auszug) . . . . . ii
A.3 Use Case-Diagramm . . . . . iii
A.4 Pflichtenheft (Auszug) . . . . . iii
A.5 Datenbankmodell . . . . . v
A.6 Oberflächenentwürfe . . . . . vi
A.7 Screenshots der Anwendung . . . . . viii
A.8 Entwicklerdokumentation . . . . . x
A.9 Testfall und sein Aufruf auf der Konsole . . . . . xii
A.10 Klasse: ComparedNaturalModuleInformation . . . . . xiii
A.11 Klassendiagramm . . . . . xvi
A.12 Benutzerdokumentation . . . . . xvii

# Abbildungsverzeichnis

1 Vereinfachtes ER-Modell . . . . . 9
2 Prozess des Einlesens eines Moduls . . . . . 10
3 Use Case-Diagramm . . . . . iii
4 Datenbankmodell . . . . . v
5 Liste der Module mit Filtermöglichkeiten . . . . . vi
6 Anzeige der Übersichtsseite einzelner Module . . . . . vii
7 Anzeige und Filterung der Module nach Tags . . . . . vii
8 Anzeige und Filterung der Module nach Tags . . . . . viii
9 Liste der Module mit Filtermöglichkeiten . . . . . ix
10 Aufruf des Testfalls auf der Konsole . . . . . xiii
11 Klassendiagramm . . . . . xvi

# Tabellenverzeichnis

1 Zeitplanung . . . . . 3
2 Entscheidungsmatrix . . . . . 8
3 Soll-/Ist-Vergleich . . . . . 13

# Listings

1 Testfall in PHP . . . . . xii
2 Klasse: ComparedNaturalModuleInformation . . . . . xiii

# Abkürzungsverzeichnis

| | |
|---|---|
| **API** | Application Programming Interface |
| **ERM** | Entity-Relationship-Modell |
| **SQL** | Structured Query Language |
| **UML** | Unified Modeling Language |

# 1 Einleitung

In dieser Dokumentation wird das Projekt 'Bereitstellung eines Microservices zur Synchronisation von dezentralisiert gespeicherten Kundendaten' beschrieben, welches im Rahmen der Prüfung zum Abschluss der Ausbildung zum Fachinformatiker Anwendungsentwicklung durchgeführt wurde. Es wird der Prozess der Entwicklung einer Anwendung dargestellt, die als eigenständiger Microservice implementiert wird.

## 1.1 Projektumfeld

JournawayGmbH ist ein in Leer ansässiges, 2017 gegründetes Reiseunternehmen und bietet Reise-komplett-Pakete auf ihrer Webseite an. Derzeit werden rund 160 Mitarbeiter beschäftigt und unter diesen Mitarbeitern arbeiten über ein Drittel in der internen IT-Abteilung, welche B2C und B2B Lösungen für das Unternehmen zur Verfügung stellt. Aufgaben dieser Abteilung liegen unteranderem darin die Webseite und die dazu gehörigen Daten, wie Buchungsdaten, Kundendaten etc. zu pflegen und zu ermöglichen, dass Preise und Informationen zu Leistungen innerhalb der Produkte stets aktuell sind.

Der Auftraggeber des Projektes ist die journawayGmbH selbst.

## 1.2 Projektziel

Journaway stellte in Auftrag ein Produkt zu entwickeln, welches als Schnittstelle Daten von zwei Kundendaten speichernden Systemen miteinander synchronisiert.

Mit der Entwicklung eines neuen Kundenportals auf der Webseite ist es notwendig, dass Informationen für die Kommunikation zwischen Unternehmen und Kunden nicht asynchron vorliegen. Teil dieser Aufgabe ist es einen Microservice zu entwickeln, der durch eventbasierte Synchronisierung die dezentrale Verfügbarkeit von Daten auflöst und eine einzige Source of Truth schafft, wodurch die Kundendaten auf beiden Systemen identisch sind.

## 1.3 Projektbegründung

In beiden bisherigen Systemen werden Kundendaten gespeichert, welche keine Relation zu einander haben und können somit für den gleichen Kunden unterschiedlich vorliegen. Mit dem neuen Microservice *Nightcrawler* sollen Fehler beim persistieren kundenkontaktrelevanter Informationen minimiert werden. Außerdem soll es ermöglichen das im bisherigen CRM *HubSpot* gespeicherte Kundendaten mit dem angemeldeten Kunden vom *user-service* verknüpft werden, wodurch im Kundenportal weitere, für den Kunden interessante Informationen dargestellt werden können.

Ohne den *Nightcrawler* müssten Mitarbeiter aus dem Kundensupport eigenhändig nach Abweichungen in Kontaktinformationen suchen und diese manuell einpflegen. Dies würde einen erhöhten Ressourcenaufwand und eine höhere Anfälligkeit für Fehler bedeuten. Ziel ist es, den Erhalt von zentralen und aktuellen Kontaktdaten, ohne manuellen Aufwand zu gewährleisten.

### 1.4 Projektschnittstellen

Die Anwendung interagiert über Event Queues durch RabbitMQ mit einem anderen Microservice *user-service*. Durch diesen Service ist es möglich, dass persönliche Informationen, Kommunikationspräferenzen oder Reisende von Websitebesuchern erstellt, bearbeitet und gelöscht werden können. Außerdem können die Nutzer dort auch ihre Buchungen einsehen.

Zudem kommuniziert die Anwendung über einen HttpClient mit dem CRM *HubSpot*. Dies ist der Ort an dem standardmäßig kundenbezogene Daten gespeichert werden. Dieser cloud-basierte Service von dem Unternehmen *HubSpot, Inc.* wird hauptsächlich für die Speicherung von Kontaktdaten, Deals und Ausführung von Workflows genutzt.

Dadurch, dass es sich um einen autonomen Synchronisationsservice handelt gibt es keine direkten Benutzer, jedoch werden events zur Synchronisierung entweder durch den Websitenutzer oder Mitarbeiter in *HubSpot* ausgelöst. Schließlich werden die Synchronisationshistorien zu Kunden in einer PostgreSql Datenbank persistiert.

Der *user-service*, als auch der *Nightcrawler* sind Microservices aufbauend auf dem .NET 10 Framework von Microsoft und werden auf Servern mit Linux basiertem Betriebssystem (Debian 12.13) ausgeführt. Der Quellcode ist in der Sprache CSharp geschrieben. Als Entwicklungsumgebung wird JetBrains Rider 2026 verwendet.

### 1.5 Projektabgrenzung

Teil dieses Projektes ist die Planung und Entwicklung eines funktionellen und testbaren Prototyps. Die Aufnahme der Anwendung in den Livebetrieb wird nicht behandelt.

## 2 Projektplanung

Um einen möglichst reibungslosen Ablauf in der Entwicklung des Microservices zu gewährleisten, braucht es eine gute Planung. In diesem Abschnitt wird daher aufgezeigt, welche Ressourcen (Zeit, Hardware) wie eingesetzt werden.

## 2.1 Projektphasen

Die Umsetzung des Projektes findet während eines vier Wochen Sprints statt und wird gemäß der Vorgaben der IHK, wie in Tabelle 1 dargestellt, innerhalb von 80 Stunden bearbeitet:

**Zeitplanung** Tabelle 1 zeigt die grobe Zeitplanung.

| Projektphase | Geplante Zeit |
|---|---|
| Analysephase | 8 h |
| Entwurfsphase | 6 h |
| Implementierungsphase | 49 h |
| Einführungsphase | 7 h |
| Erstellen der Dokumentation | 9 h |
| Dokumentationsphase | 10 h |
| **Gesamt** | **80 h** |

Tabelle 1: Zeitplanung

Eine detailliertere Zeitplanung findet sich im Anhang A.1: Detaillierte Zeitplanung auf Seite i.

## 2.2 Abweichungen vom Projektantrag

Bei dem Inhalt des Projektes wird die Anbindung von Hangfire mit der Anbindung von RabbitMQ ausgetauscht, da bei der Planung erkannt wurde, dass Hangfire in der ersten Iteration nicht benötigt wird und sich bei der Umsetzung der Event Queues auf den Message Broker RabbitMQ geeinigt wurde.

## 2.3 Ressourcenplanung

**Hardware**

1. Büroarbeitsplatz
2. Netzwerk mit Internetzugang
3. Laptop und zwei weitere Bildschirme
4. Server für die Anwendung und Datenbank

**Software**

1. Betriebssystem Laptop: Windows 11
2. Entwicklungsumgebung: JetBrains Rider 2026
3. Message Broker Software: RabbitMQ

4. Datenbanksoftware: PostgreSQL 16
5. Zielbetriebssystem: Linux Debian 12.13
6. Software zum generieren und einsehen von Testdaten auf der Datenbank: HeidiSQL

**Personal**

1. Für die Qualitätssicherung während der Review- und Testphase werden zwei weitere Entwickler benötigt.

### 2.4 Entwicklungsprozess

Während der Umsetzung des Projektes wird ein agiler Prozess bzw. Scrum verfolgt. Dies ist ein Prozess, bei dem iterativ kleinere Abschnitte eines Auftrages fertiggestellt werden, wodurch schnell auf unvorhergesehene Probleme oder neue Anforderungen eingegangen werden kann. Scrum zeichnet sich durch eine hohe Flexibilität aus und eignet sich gut für kleine Projekte bzw. Teams.

## 3 Analysephase

Im Folgenden wird zunächst die bisherige Situation im Unternehmen betrachtet. Darauf aufbauend wird untersucht, welchen wirtschaftlichen Nutzen das Projekt bietet. Anschließend werden die fachlichen und technischen Anforderungen an die Anwendung beschrieben. Zum Abschluss folgt eine Nutzwertanalyse, um den nicht-monetären Nutzen des Projektes zu bewerten.

### 3.1 Ist-Analyse

Aktuell werden kundenbezogene Daten in zwei voneinander getrennten Systemen gespeichert. Zum einen im *user-service*, welcher Daten für das Kundenportal verwaltet, und zum anderen im CRM-System *HubSpot*. Zwischen beiden Systemen besteht bislang keine eigenständig verwaltete und durchgängig nachvollziehbare Synchronisation. Dadurch kann es passieren, dass Kontaktinformationen eines Kunden in beiden Systemen unterschiedlich vorliegen.

Besonders problematisch ist dies bei Informationen, die für die Kommunikation mit dem Kunden relevant sind. Dazu zählen unter anderem die E-Mail-Adresse, der Vorname, der Nachname, die Anrede sowie Kommunikationspräferenzen. Werden diese Daten in einem der beiden Systeme geändert, ist nicht sichergestellt, dass die Änderung zuverlässig im jeweils anderen System übernommen wird.

Zusätzlich existiert bislang keine zentrale Stelle, die nachvollziehbar speichert, wann eine Synchronisation durchgeführt wurde, aus welchem System eine Änderung stammt und ob diese

erfolgreich verarbeitet wurde. Fehler bei der Übertragung könnten dadurch nur mit erhöhtem manuellem Aufwand erkannt und korrigiert werden. Ebenso gibt es bisher keine eigenständige technische Lösung, um mit temporären Fehlern, Systemausfällen oder Rate Limits der HubSpot API umzugehen.

Aus diesem Grund soll mit dem Projekt *Nightcrawler* ein eigenständiger Microservice entwickelt werden, der die Synchronisation zwischen beiden Systemen übernimmt. Hierzu soll eine PostgreSQL-Datenbank eingeführt werden, in der sowohl der aktuelle Synchronisationszustand zwischen Benutzer und Kontakt, als auch die Historie aller Synchronisationsereignisse gespeichert wird.

Außerdem soll die Kommunikation zwischen den beteiligten Systemen asynchron über RabbitMQ erfolgen. Dafür werden drei Event Queues benötigt: eine Queue für Änderungen vom *user-service* in Richtung *HubSpot*, eine Queue für Änderungen von *HubSpot* in Richtung *user-service* sowie eine weitere Queue, über die der *user-service* den erfolgreichen Abschluss einer Synchronisation an den *Nightcrawler* zurückmelden kann.

Die bisherige Situation soll also dahingehend verbessert werden, dass Kundendaten zwischen beiden Systemen automatisch, nachvollziehbar und fehlertolerant synchronisiert werden können. Zusätzlich soll die neue Anwendung in der Lage sein, Konflikte bei gleichzeitigen Änderungen zu erkennen und nach festgelegten Regeln aufzulösen.

## 3.2 Wirtschaftlichkeitsanalyse

Die Entwicklung des Microservices verursacht zunächst einmalige Projektkosten in Form von Arbeitszeit. Für die Umsetzung des Projektes sind insgesamt 80 Stunden vorgesehen. Bei einem internen Verrechnungssatz von beispielsweise [70] Euro pro Stunde ergeben sich folgende Entwicklungskosten:

$$80 \text{ Stunden} \times [70] \text{ Euro/Stunde} = [5600] \text{ Euro}$$

Dem gegenüber steht der Nutzen, der sich aus der automatisierten Synchronisation von Kundendaten ergibt. Ohne eine solche Lösung müssten Abweichungen zwischen den Datenbeständen manuell erkannt, überprüft und korrigiert werden. Dies würde insbesondere im Kundensupport und in der technischen Betreuung zusätzlichen Aufwand verursachen.

Zusätzlich sinkt durch die Anwendung das Risiko, dass veraltete oder fehlerhafte Kontaktdaten in einem der Systeme verwendet werden. Dadurch können Fehler in Folgeprozessen reduziert werden, beispielsweise bei der Kundenkommunikation oder bei der Darstellung von Informationen im Kundenportal.

Wird angenommen, dass durch die automatisierte Synchronisation wöchentlich [2 bis 3] Stunden manueller Aufwand eingespart werden können, ergibt sich bei einem internen Stundensatz von [40] Euro ein jährliches Einsparpotenzial von:

$$[2{,}5] \text{ Stunden/Woche} \times [40] \text{ Euro/Stunde} \times [52] \text{ Wochen} = [5200] \text{ Euro/Jahr}$$

Neben dieser direkten Einsparung entsteht zusätzlicher Nutzen durch eine höhere Datenqualität, bessere Nachvollziehbarkeit und eine geringere Fehleranfälligkeit. Das Projekt ist damit wirtschaftlich sinnvoll, da die Entwicklungskosten mittelfristig durch eingesparte Arbeitszeit und verbesserte Prozesse ausgeglichen werden können.

## 3.3 Anforderungsanalyse

Damit das Projektziel erreicht werden kann, muss die Anwendung sowohl fachliche als auch technische Anforderungen erfüllen.

Fachlich muss der *Nightcrawler* in der Lage sein, Änderungen an Kundendaten zwischen dem *user-service* und *HubSpot* in beide Richtungen zu synchronisieren. Dabei sollen mindestens die Datenfelder *email*, *givenName*, *surname*, *salutation* und *CommunicationPreference* berücksichtigt werden.

Wenn ein Benutzer im *user-service* erstellt oder geändert wird, soll ein entsprechendes Ereignis verarbeitet und die dazugehörigen Daten in *HubSpot* aktualisiert werden. Wird ein Benutzer neu erstellt, soll zunächst geprüft werden, ob anhand der E-Mail-Adresse bereits ein passender Kontakt in *HubSpot* existiert. Ist dies der Fall, soll dieser Kontakt aktualisiert werden. Falls kein Kontakt gefunden wird, soll ein neuer Kontakt erstellt werden.

Wenn dagegen in *HubSpot* ein Kontakt geändert wird, soll diese Änderung ebenfalls verarbeitet und an den *user-service* weitergegeben werden. Kontakte aus *HubSpot*, für die kein passender Benutzer im *user-service* existiert, sollen zunächst ignoriert werden.

Damit jede Synchronisation nachvollziehbar bleibt, muss jedes verarbeitete Ereignis zunächst in einer Historie gespeichert werden. Diese *SyncHistory* soll Informationen zur Quelle des Ereignisses, zum Ereignistyp, zu Benutzer- und Kontaktzuordnung, zum Payload, zum Zeitpunkt des Auftretens, zum aktuellen Status sowie zur Anzahl der bisherigen Verarbeitungsversuche enthalten.

Zusätzlich muss ein *UserContactState* gespeichert werden. Dieser dient dazu, die Zuordnung zwischen Benutzer und HubSpot-Kontakt sowie den letzten bekannten Synchronisationsstand zu verwalten. Dabei sollen unter anderem die Zeitpunkte der letzten Änderung im *user-service*, der letzten Änderung in *HubSpot* und der letzten erfolgreichen Synchronisation gespeichert werden.

Technisch muss die Anwendung drei Event Queues über RabbitMQ verwenden. Da keine Nachrichten bei einem Neustart oder Ausfall verloren gehen sollen, müssen die Hauptqueues als durable Quorum Queues umgesetzt werden. Für temporäre Fehler soll eine Retry-Logik mit TTL-basierten Retry Queues eingesetzt werden. Nicht behandelbare Fehler sollen in einer Dead-Letter-Queue abgelegt werden.

Zur Konfliktbehandlung soll eine optimistische Strategie verwendet werden. Wenn Daten in beiden Systemen unterschiedlich geändert wurden, soll der Datenstand des *user-service* bevorzugt werden. Wird also eine Änderung aus *HubSpot* erkannt, die im Konflikt mit aktuelleren Daten aus dem *user-service* steht, soll diese Änderung nicht übernommen, sondern stattdessen eine neue Synchronisation in Richtung *HubSpot* angestoßen werden.

## 3.4 Nutzwertanalyse

Neben dem wirtschaftlichen Nutzen bietet das Projekt auch einen deutlichen nicht-monetären Mehrwert. Insbesondere die Datenqualität, die Nachvollziehbarkeit von Änderungen und die technische Stabilität der Synchronisation werden verbessert.

Vor der Umsetzung des Projektes besteht das Problem, dass Änderungen an Kundendaten nicht zentral nachvollzogen werden können. Außerdem ist die Synchronisation zwischen beiden Systemen nicht eigenständig abgesichert, wodurch Fehler bei der Übertragung schwerer zu erkennen sind. Mit dem *Nightcrawler* wird eine Lösung geschaffen, die Änderungen dokumentiert, Synchronisationszustände speichert und Fehlerfälle gezielt behandeln kann.

Zur Bewertung des Nutzwertes werden die Kriterien Datenkonsistenz, Nachvollziehbarkeit, Ausfallsicherheit, Wartbarkeit und Automatisierungsgrad betrachtet. Die folgende Matrix stellt einen beispielhaften Vorher-/Nachher-Vergleich dar:

| **Kriterium** | **Gewichtung** | **Vorher** | **Nachher** |
|---|---|---|---|
| Datenkonsistenz | [30] | [2] | [5] |
| Nachvollziehbarkeit | [20] | [1] | [5] |
| Ausfallsicherheit | [20] | [2] | [4] |
| Wartbarkeit | [15] | [2] | [4] |
| Automatisierungsgrad | [15] | [2] | [5] |

Aus den beispielhaften Bewertungen ergibt sich, dass der Nutzen der neuen Anwendung vor allem in der zuverlässigeren Verarbeitung, der besseren Transparenz und der geringeren manuellen Abhängigkeit liegt.

Das Projekt verbessert somit nicht nur die technische Grundlage für die Synchronisation von Kundendaten, sondern schafft auch eine stabilere und besser wartbare Lösung für zukünftige Erweiterungen.

**Beispiel** Ein Beispiel für eine Entscheidungsmatrix findet sich in Kapitel 4.2: Architekturdesign.

# 4 Entwurfsphase

## 4.1 Zielplattform

- Beschreibung der Kriterien zur Auswahl der Zielplattform (u. a. Programmiersprache, Datenbank, Client/Server, Hardware).

## 4.2 Architekturdesign

- Beschreibung und Begründung der gewählten Anwendungsarchitektur (z. B. **MVC!**).
- Ggfs. Bewertung und Auswahl von verwendeten Frameworks sowie ggfs. eine kurze Einführung in die Funktionsweise des verwendeten Frameworks.

**Beispiel** Anhand der Entscheidungsmatrix in Tabelle 2 wurde für die Implementierung der Anwendung das **PHP!**-Framework Symfony[1] ausgewählt.

| Eigenschaft | Gewichtung | Akelos | CakePHP | Symfony | Eigenentwicklung |
|---|---|---|---|---|---|
| **Dokumentation** | 5 | 4 | 3 | 5 | 0 |
| **Reenginierung** | 3 | 4 | 2 | 5 | 3 |
| **Generierung** | 3 | 5 | 5 | 5 | 2 |
| **Testfälle** | 2 | 3 | 2 | 3 | 3 |
| **Standardaufgaben** | 4 | 3 | 3 | 3 | 0 |
| **Gesamt:** | **17** | **65** | **52** | **73** | **21** |
| **Nutzwert:** | | **3,82** | **3,06** | **4,29** | **1,24** |

Tabelle 2: Entscheidungsmatrix

## 4.3 Entwurf der Benutzeroberfläche

- Entscheidung für die gewählte Benutzeroberfläche (z. B. GUI, Webinterface).
- Beschreibung des visuellen Entwurfs der konkreten Oberfläche (z. B. Mockups, Menüführung).
- Ggfs. Erläuterung von angewendeten Richtlinien zur Usability und Verweis auf Corporate Design.

[1] Vgl. **?**.

**Beispiel** Beispielentwürfe finden sich im Anhang A.6: Oberflächenentwürfe auf Seite vi.

## 4.4 Datenmodell

- Entwurf/Beschreibung der Datenstrukturen (z. B. ERM und/oder Tabellenmodell, **XML!**-Schemas) mit kurzer Beschreibung der wichtigsten (!) verwendeten Entitäten.

**Beispiel** In Abbildung 1 wird ein Entity-Relationship-Modell (ERM) dargestellt, welches lediglich Entitäten, Relationen und die dazugehörigen Kardinalitäten enthält.

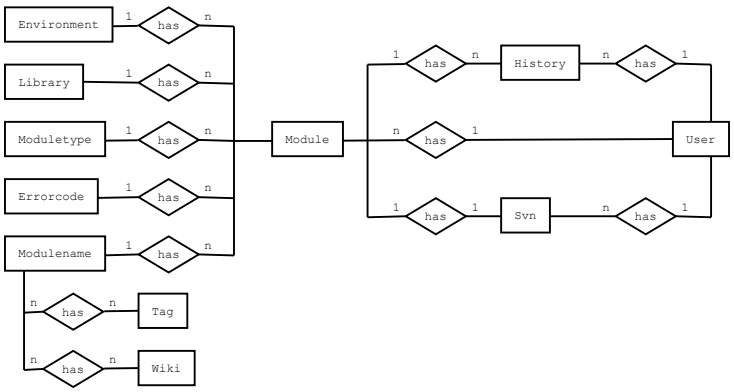


Abbildung 1: Vereinfachtes ER-Modell

## 4.5 Geschäftslogik

- Modellierung und Beschreibung der wichtigsten (!) Bereiche der Geschäftslogik (z. B. mit Komponenten-, Klassen-, Sequenz-, Datenflussdiagramm, Programmablaufplan, Struktogramm, **EPK!** (**EPK!**)).
- Wie wird die erstellte Anwendung in den Arbeitsfluss des Unternehmens integriert?

**Beispiel** Ein Klassendiagramm, welches die Klassen der Anwendung und deren Beziehungen untereinander darstellt kann im Anhang A.11: Klassendiagramm auf Seite xvi eingesehen werden.

Abbildung 2 zeigt den grundsätzlichen Programmablauf beim Einlesen eines Moduls als **EPK!**.

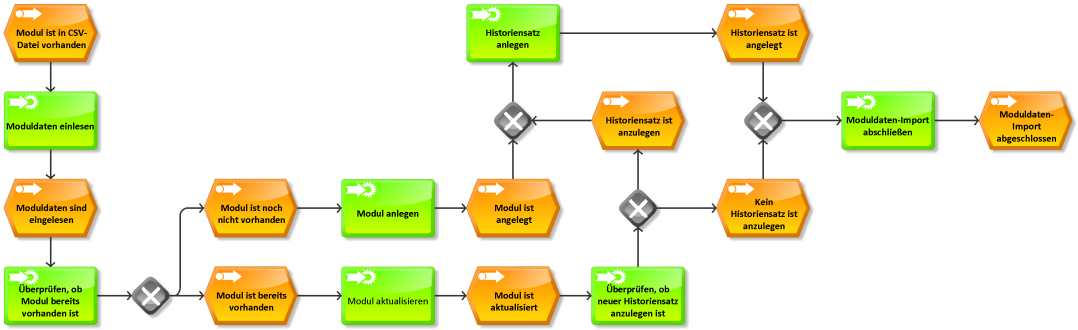


Abbildung 2: Prozess des Einlesens eines Moduls

### 4.6 Maßnahmen zur Qualitätssicherung

- Welche Maßnahmen werden ergriffen, um die Qualität des Projektergebnisses (siehe Kapitel **??**: **??**) zu sichern (z. B. automatische Tests, Anwendertests)?
- Ggfs. Definition von Testfällen und deren Durchführung (durch Programme/Benutzer).

### 4.7 Pflichtenheft/Datenverarbeitungskonzept

- Auszüge aus dem Pflichtenheft/Datenverarbeitungskonzept, wenn es im Rahmen des Projekts erstellt wurde.

**Beispiel** Ein Beispiel für das auf dem Lastenheft (siehe Kapitel **??**: **??**) aufbauende Pflichtenheft ist im Anhang A.4: Pflichtenheft (Auszug) auf Seite iii zu finden.

## 5 Implementierungsphase

### 5.1 Implementierung der Datenstrukturen

- Beschreibung der angelegten Datenbank (z. B. Generierung von SQL aus Modellierungswerkzeug oder händisches Anlegen), **XML!**-Schemas usw..

### 5.2 Implementierung der Benutzeroberfläche

- Beschreibung der Implementierung der Benutzeroberfläche, falls dies separat zur Implementierung der Geschäftslogik erfolgt (z. B. bei **HTML!**-Oberflächen und Stylesheets).
- Ggfs. Beschreibung des Corporate Designs und dessen Umsetzung in der Anwendung.
- Screenshots der Anwendung

**Beispiel** Screenshots der Anwendung in der Entwicklungsphase mit Dummy-Daten befinden sich im Anhang A.7: Screenshots der Anwendung auf Seite viii.

### 5.3 Implementierung der Geschäftslogik

- Beschreibung des Vorgehens bei der Umsetzung/Programmierung der entworfenen Anwendung.
- Ggfs. interessante Funktionen/Algorithmen im Detail vorstellen, verwendete Entwurfsmuster zeigen.
- Quelltextbeispiele zeigen.
- Hinweis: Wie in Kapitel 1: Einleitung zitiert, wird nicht ein lauffähiges Programm bewertet, sondern die Projektdurchführung. Dennoch würde ich immer Quelltextausschnitte zeigen, da sonst Zweifel an der tatsächlichen Leistung des Prüflings aufkommen können.

**Beispiel** Die Klasse `ComparedNaturalModuleInformation` findet sich im Anhang A.10: Klasse: ComparedNaturalModuleInformation auf Seite xiii.

## 6 Abnahmephase

- Welche Tests (z. B. Unit-, Integrations-, Systemtests) wurden durchgeführt und welche Ergebnisse haben sie geliefert (z. B. Logs von Unit Tests, Testprotokolle der Anwender)?
- Wurde die Anwendung offiziell abgenommen?

**Beispiel** Ein Auszug eines Unit Tests befindet sich im Anhang A.9: Testfall und sein Aufruf auf der Konsole auf Seite xii. Dort ist auch der Aufruf des Tests auf der Konsole des Webservers zu sehen.

## 7 Einführungsphase

- Welche Schritte waren zum Deployment der Anwendung nötig und wie wurden sie durchgeführt (automatisiert/manuell)?
- Wurden ggfs. Altdaten migriert und wenn ja, wie?
- Wurden Benutzerschulungen durchgeführt und wenn ja, Wie wurden sie vorbereitet?

# 8 Dokumentation

- Wie wurde die Anwendung für die Benutzer/Administratoren/Entwickler dokumentiert (z. B. Benutzerhandbuch, API-Dokumentation)?
- Hinweis: Je nach Zielgruppe gelten bestimmte Anforderungen für die Dokumentation (z. B. keine IT-Fachbegriffe in einer Anwenderdokumentation verwenden, aber auf jeden Fall in einer Dokumentation für den IT-Bereich).

**Beispiel** Ein Ausschnitt aus der erstellten Benutzerdokumentation befindet sich im Anhang A.12: Benutzerdokumentation auf Seite xvii. Die Entwicklerdokumentation wurde mittels PHPDoc[2] automatisch generiert. Ein beispielhafter Auszug aus der Dokumentation einer Klasse findet sich im Anhang A.8: Entwicklerdokumentation auf Seite x.

# 9 Fazit

## 9.1 Soll-/Ist-Vergleich

- Wurde das Projektziel erreicht und wenn nein, warum nicht?
- Ist der Auftraggeber mit dem Projektergebnis zufrieden und wenn nein, warum nicht?
- Wurde die Projektplanung (Zeit, Kosten, Personal, Sachmittel) eingehalten oder haben sich Abweichungen ergeben und wenn ja, warum?
- Hinweis: Die Projektplanung muss nicht strikt eingehalten werden. Vielmehr sind Abweichungen sogar als normal anzusehen. Sie müssen nur vernünftig begründet werden (z. B. durch Änderungen an den Anforderungen, unter-/überschätzter Aufwand).

**Beispiel (verkürzt)** Wie in Tabelle 3 zu erkennen ist, konnte die Zeitplanung bis auf wenige Ausnahmen eingehalten werden.

## 9.2 Lessons Learned

- Was hat der Prüfling bei der Durchführung des Projekts gelernt (z. B. Zeitplanung, Vorteile der eingesetzten Frameworks, Änderungen der Anforderungen)?

[2] Vgl. **?**

| Phase | Geplant | Tatsächlich | Differenz |
|---|---|---|---|
| **Entwurfsphase** | 19 h | 19 h | |
| **Analysephase** | 9 h | 10 h | +1 h |
| **Implementierungsphase** | 29 h | 28 h | -1 h |
| **Abnahmetest der Fachabteilung** | 1 h | 1 h | |
| **Einführungsphase** | 1 h | 1 h | |
| **Erstellen der Dokumentation** | 9 h | 11 h | +2 h |
| **Pufferzeit** | 2 h | 0 h | -2 h |
| **Gesamt** | 70 h | 70 h | |

Tabelle 3: Soll-/Ist-Vergleich

## 9.3 Ausblick

- Wie wird sich das Projekt in Zukunft weiterentwickeln (z. B. geplante Erweiterungen)?

# Literaturverzeichnis

## Eidesstattliche Erklärung

Ich, Niklas Hamburg, versichere hiermit, dass ich meine **Dokumentation zur betrieblichen Projektarbeit** mit dem Thema

> *Bereitstellung eines Microservices – zur Synchronisation von dezentralisiert gespeicherten Kundendaten*

selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, wobei ich alle wörtlichen und sinngemäßen Zitate als solche gekennzeichnet habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Papenburg, den 08.05.2026

Niklas Hamburg

# A Anhang

## A.1 Detaillierte Zeitplanung

| | | | |
|---|---|---|---|
| **Analysephase** | | | **9 h** |
| 1. Analyse des Ist-Zustands | | 3 h | |
| 1.1. Fachgespräch mit der EDV-Abteilung | 1 h | | |
| 1.2. Prozessanalyse | 2 h | | |
| 2. „Make or buy“-Entscheidung und Wirtschaftlichkeitsanalyse | | 1 h | |
| 3. Erstellen eines „Use-Case“-Diagramms | | 2 h | |
| 4. Erstellen des Lastenhefts mit der EDV-Abteilung | | 3 h | |
| **Entwurfsphase** | | | **19 h** |
| 1. Prozessentwurf | | 2 h | |
| 2. Datenbankentwurf | | 3 h | |
| 2.1. ER-Modell erstellen | 2 h | | |
| 2.2. Konkretes Tabellenmodell erstellen | 1 h | | |
| 3. Erstellen von Datenverarbeitungskonzepten | | 4 h | |
| 3.1. Verarbeitung der CSV-Daten | 1 h | | |
| 3.2. Verarbeitung der SVN-Daten | 1 h | | |
| 3.3. Verarbeitung der Sourcen der Programme | 2 h | | |
| 4. Benutzeroberflächen entwerfen und abstimmen | | 2 h | |
| 5. Erstellen eines UML-Komponentendiagramms der Anwendung | | 4 h | |
| 6. Erstellen des Pflichtenhefts | | 4 h | |
| **Implementierungsphase** | | | **29 h** |
| 1. Anlegen der Datenbank | | 1 h | |
| 2. Umsetzung der HTML-Oberflächen und Stylesheets | | 4 h | |
| 3. Programmierung der PHP-Module für die Funktionen | | 23 h | |
| 3.1. Import der Modulinformationen aus CSV-Dateien | 2 h | | |
| 3.2. Parsen der Modulquelltexte | 3 h | | |
| 3.3. Import der SVN-Daten | 2 h | | |
| 3.4. Vergleichen zweier Umgebungen | 4 h | | |
| 3.5. Abrufen der von einem zu wählenden Benutzer geänderten Module | 3 h | | |
| 3.6. Erstellen einer Liste der Module unter unterschiedlichen Aspekten | 5 h | | |
| 3.7. Anzeigen einer Liste mit den Modulen und geparsten Metadaten | 3 h | | |
| 3.8. Erstellen einer Übersichtsseite für ein einzelnes Modul | 1 h | | |
| 4. Nächtlichen Batchjob einrichten | | 1 h | |
| **Abnahmetest der Fachabteilung** | | | **1 h** |
| 1. Abnahmetest der Fachabteilung | | 1 h | |
| **Einführungsphase** | | | **1 h** |
| 1. Einführung/Benutzerschulung | | 1 h | |
| **Erstellen der Dokumentation** | | | **9 h** |
| 1. Erstellen der Benutzerdokumentation | | 2 h | |
| 2. Erstellen der Projektdokumentation | | 6 h | |
| 3. Programmdokumentation | | 1 h | |
| 3.1. Generierung durch PHPdoc | 1 h | | |
| **Pufferzeit** | | | **2 h** |
| 1. Puffer | | 2 h | |
| **Gesamt** | | | **70 h** |

## A.2 Lastenheft (Auszug)

Es folgt ein Auszug aus dem Lastenheft mit Fokus auf die Anforderungen:

Die Anwendung muss folgende Anforderungen erfüllen:

1. Verarbeitung der Moduldaten
   1.1. Die Anwendung muss die von Subversion und einem externen Programm bereitgestellten Informationen (z.B. Source-Benutzer, -Datum, Hash) verarbeiten.
   1.2. Auslesen der Beschreibung und der Stichwörter aus dem Sourcecode.
2. Darstellung der Daten
   2.1. Die Anwendung muss eine Liste aller Module erzeugen inkl. Source-Benutzer und -Datum, letztem Commit-Benutzer und -Datum für alle drei Umgebungen.
   2.2. Verknüpfen der Module mit externen Tools wie z.B. Wiki-Einträgen zu den Modulen oder dem Sourcecode in Subversion.
   2.3. Die Sourcen der Umgebungen müssen verglichen und eine schnelle Übersicht zur Einhaltung des allgemeinen Entwicklungsprozesses gegeben werden.
   2.4. Dieser Vergleich muss auf die von einem bestimmten Benutzer bearbeiteten Module eingeschränkt werden können.
   2.5. Die Anwendung muss in dieser Liste auch Module anzeigen, die nach einer Bearbeitung durch den gesuchten Benutzer durch jemand anderen bearbeitet wurden.
   2.6. Abweichungen sollen kenntlich gemacht werden.
   2.7. Anzeigen einer Übersichtsseite für ein Modul mit allen relevanten Informationen zu diesem.
3. Sonstige Anforderungen
   3.1. Die Anwendung muss ohne das Installieren einer zusätzlichen Software über einen Webbrowser im Intranet erreichbar sein.
   3.2. Die Daten der Anwendung müssen jede Nacht bzw. nach jedem **SVN!**-Commit automatisch aktualisiert werden.
   3.3. Es muss ermittelt werden, ob Änderungen auf der Produktionsumgebung vorgenommen wurden, die nicht von einer anderen Umgebung kopiert wurden. Diese Modulliste soll als Mahnung per E-Mail an alle Entwickler geschickt werden (Peer Pressure).
   3.4. Die Anwendung soll jederzeit erreichbar sein.
   3.5. Da sich die Entwickler auf die Anwendung verlassen, muss diese korrekte Daten liefern und darf keinen Interpretationsspielraum lassen.
   3.6. Die Anwendung muss so flexibel sein, dass sie bei Änderungen im Entwicklungsprozess einfach angepasst werden kann.

## A.3 Use Case-Diagramm

Use Case-Diagramme und weitere UML-Diagramme kann man auch direkt mit LaTeX zeichnen, siehe z. B. http://metauml.sourceforge.net/old/usecase-diagram.html.

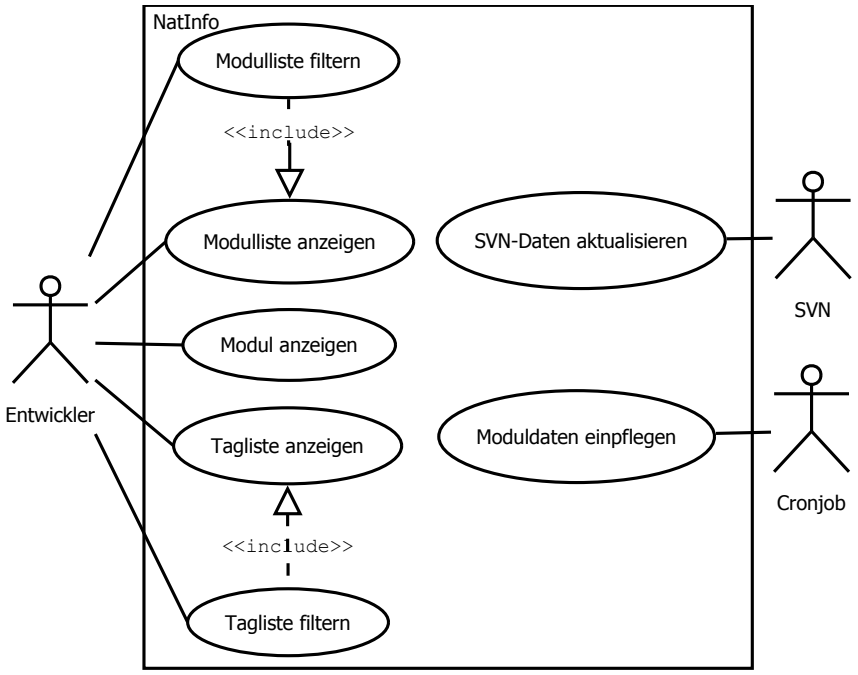


Abbildung 3: Use Case-Diagramm

## A.4 Pflichtenheft (Auszug)

### Zielbestimmung

1. Musskriterien

   1.1. Modul-Liste: Zeigt eine filterbare Liste der Module mit den dazugehörigen Kerninformationen sowie Symbolen zur Einhaltung des Entwicklungsprozesses an

   - In der Liste wird der Name, die Bibliothek und Daten zum Source und Kompilat eines Moduls angezeigt.
   - Ebenfalls wird der Status des Moduls hinsichtlich Source und Kompilat angezeigt. Dazu gibt es unterschiedliche Status-Zeichen, welche symbolisieren in wie weit der Entwicklungsprozess eingehalten wurde bzw. welche Schritte als nächstes getan werden müssen. So gibt es z. B. Zeichen für das Einhalten oder Verletzen des Prozesses oder den Hinweis auf den nächsten zu tätigenden Schritt.
   - Weiterhin werden die Benutzer und Zeitpunkte der aktuellen Version der Sourcen und Kompilate angezeigt. Dazu kann vorher ausgewählt werden, von welcher Umgebung diese Daten gelesen werden sollen.

- Es kann eine Filterung nach allen angezeigten Daten vorgenommen werden. Die Daten zu den Sourcen sind historisiert. Durch die Filterung ist es möglich, auch Module zu finden, die in der Zwischenzeit schon von einem anderen Benutzer editiert wurden.

1.2. Tag-Liste: Bietet die Möglichkeit die Module anhand von Tags zu filtern.
- Es sollen die Tags angezeigt werden, nach denen bereits gefiltert wird und die, die noch der Filterung hinzugefügt werden könnten, ohne dass die Ergebnisliste leer wird.
- Zusätzlich sollen die Module angezeigt werden, die den Filterkriterien entsprechen. Sollten die Filterkriterien leer sein, werden nur die Module angezeigt, welche mit einem Tag versehen sind.

1.3. Import der Moduldaten aus einer bereitgestellten **CSV!**-Datei
- Es wird täglich eine Datei mit den Daten der aktuellen Module erstellt. Diese Datei wird (durch einen Cronjob) automatisch nachts importiert.
- Dabei wird für jedes importierte Modul ein Zeitstempel aktualisiert, damit festgestellt werden kann, wenn ein Modul gelöscht wurde.
- Die Datei enthält die Namen der Umgebung, der Bibliothek und des Moduls, den Programmtyp, den Benutzer und Zeitpunkt des Sourcecodes sowie des Kompilats und den Hash des Sourcecodes.
- Sollte sich ein Modul verändert haben, werden die entsprechenden Daten in der Datenbank aktualisiert. Die Veränderungen am Source werden dabei aber nicht ersetzt, sondern historisiert.

1.4. Import der Informationen aus **SVN!** (**SVN!**). Durch einen „post-commit-hook“ wird nach jedem Einchecken eines Moduls ein **PHP!**-Script auf der Konsole aufgerufen, welches die Informationen, die vom **SVN!**-Kommandozeilentool geliefert werden, an **NatInfo!** übergibt.

1.5. Parsen der Sourcen
- Die Sourcen der Entwicklungsumgebung werden nach Tags, Links zu Artikeln im Wiki und Programmbeschreibungen durchsucht.
- Diese Daten werden dann entsprechend angelegt, aktualisiert oder nicht mehr gesetzte Tags/Wikiartikel entfernt.

1.6. Sonstiges
- Das Programm läuft als Webanwendung im Intranet.
- Die Anwendung soll möglichst leicht erweiterbar sein und auch von anderen Entwicklungsprozessen ausgehen können.
- Eine Konfiguration soll möglichst in zentralen Konfigurationsdateien erfolgen.

### Produkteinsatz

1. Anwendungsbereiche
   Die Webanwendung dient als Anlaufstelle für die Entwicklung. Dort sind alle Informationen für die Module an einer Stelle gesammelt. Vorher getrennte Anwendungen werden ersetzt bzw. verlinkt.
2. Zielgruppen
   wird lediglich von den **Natural!** (**Natural!**)-Entwicklern in der EDV-Abteilung genutzt.
3. Betriebsbedingungen
   Die nötigen Betriebsbedingungen, also der Webserver, die Datenbank, die Versionsverwaltung, das Wiki und der nächtliche Export sind bereits vorhanden und konfiguriert. Durch einen täglichen Cronjob werden entsprechende Daten aktualisiert, die Webanwendung ist jederzeit aus dem Intranet heraus erreichbar.

## A.5 Datenbankmodell

ER-Modelle kann man auch direkt mit LaTeX zeichnen, siehe z. B. http://www.texample.net/tikz/examples/entity-relationship-diagram/.

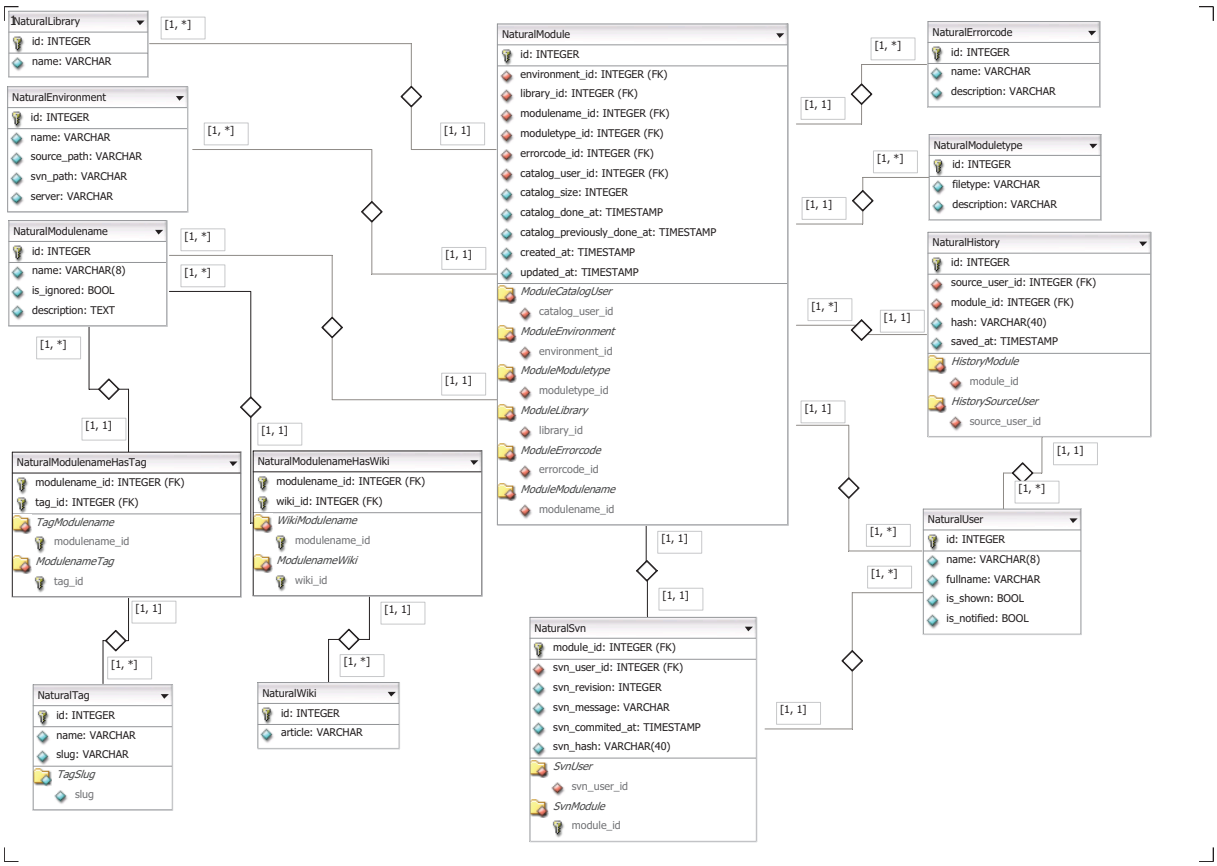


Abbildung 4: Datenbankmodell

## A.6 Oberflächenentwürfe

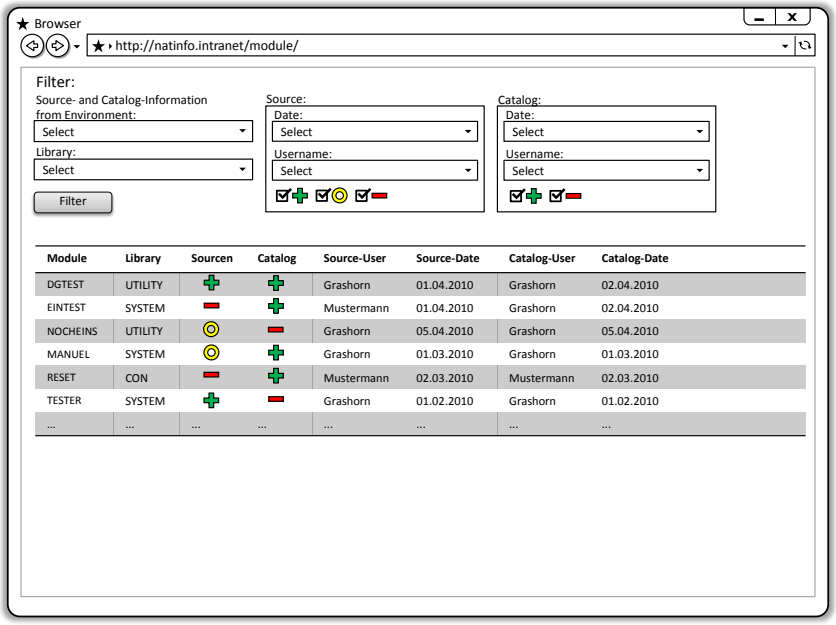


Abbildung 5: Liste der Module mit Filtermöglichkeiten

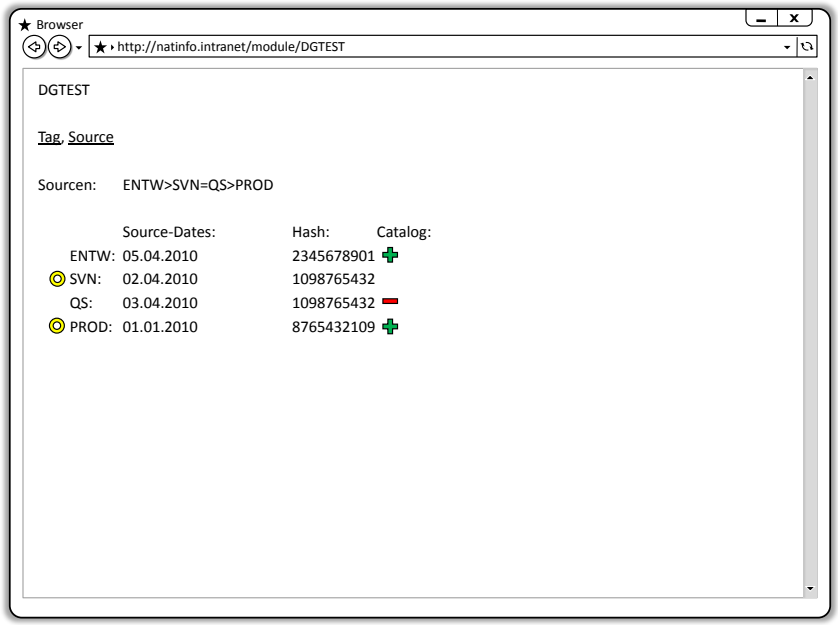


Abbildung 6: Anzeige der Übersichtsseite einzelner Module

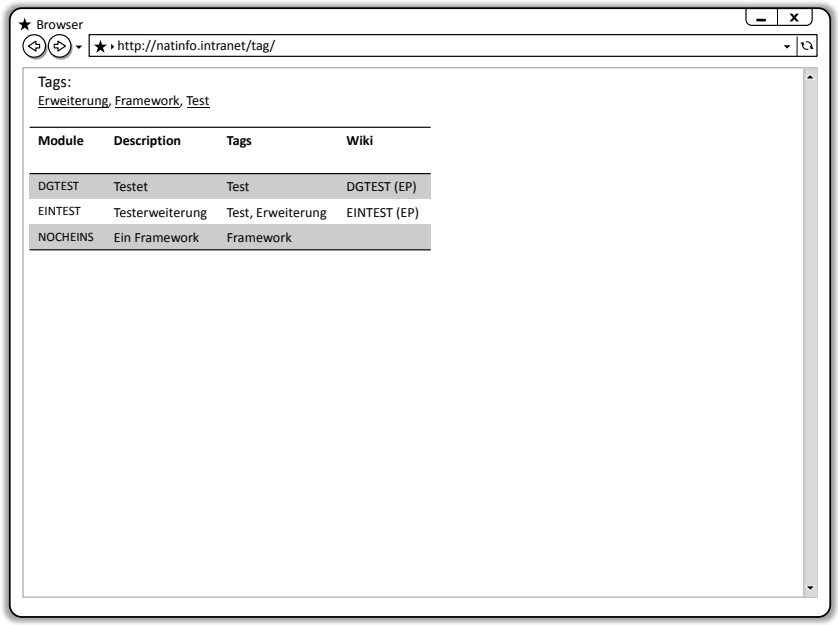


Abbildung 7: Anzeige und Filterung der Module nach Tags

## A.7 Screenshots der Anwendung

| Modulename | Description | Tags | Wiki |
|---|---|---|---|
| DGTEST | Macht einen ganz tollen Tab. | HGP | SMTAB_(EP), b |
| MALWAS | | HGP, Test | |
| HDRGE | | HGP, Project | |
| WURAM | | HGP, Test | |
| PAMIU | | HGP | |

Abbildung 8: Anzeige und Filterung der Module nach Tags

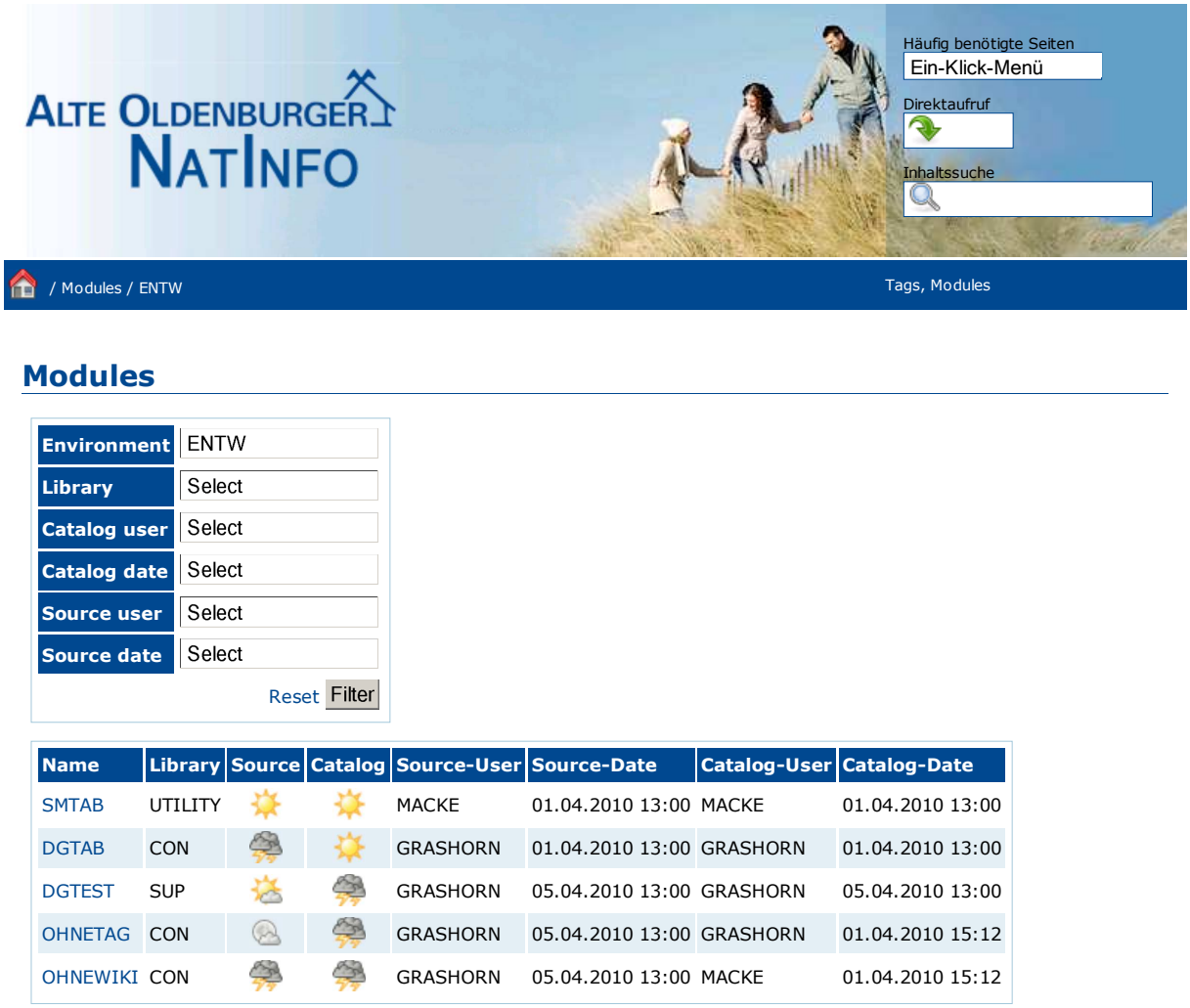


Abbildung 9: Liste der Module mit Filtermöglichkeiten

## A.8 Entwicklerdokumentation

**lib-model**

[ class tree: lib-model ] [ index: lib-model ] [ all elements ]

**Packages:**
lib-model

**Files:**
Naturalmodulename.php

**Classes:**
Naturalmodulename

# Class: Naturalmodulename

Source Location: /Naturalmodulename.php

### Class Overview

```
BaseNaturalmodulename
    |
    --Naturalmodulename
```

Subclass for representing a row from the 'NaturalModulename' table.

### Methods

- __construct
- getNaturalTags
- getNaturalWikis
- loadNaturalModuleInformation
- __toString

---

### Class Details

[line 10]
Subclass for representing a row from the 'NaturalModulename' table.

Adds some business logic to the base.

[ Top ]

---

### Class Methods

---

**constructor __construct [line 56]**

```
Naturalmodulename __construct( )
```

Initializes internal state of Naturalmodulename object.

**Tags:**

**see:** parent::__construct()
**access:** public

[ Top ]

---

**method getNaturalTags [line 68]**

```
array getNaturalTags( )
```

Returns an Array of NaturalTags connected with this Modulename.

**Tags:**

**return:** Array of NaturalTags
**access:** public

[ Top ]

### method getNaturalWikis [line 83]

```
array getNaturalWikis( )
```

Returns an Array of NaturalWikis connected with this Modulename.

**Tags:**

**return:** Array of NaturalWikis
**access:** public

[ Top ]

### method loadNaturalModuleInformation [line 17]

```
ComparedNaturalModuleInformation
loadNaturalModuleInformation( )
```

Gets the ComparedNaturalModuleInformation for this NaturalModulename.

**Tags:**

**access:** public

[ Top ]

### method __toString [line 47]

```
string __toString( )
```

Returns the name of this NaturalModulename.

**Tags:**

**access:** public

[ Top ]

Documentation generated on Thu, 22 Apr 2010 08:14:01 +0200 by phpDocumentor 1.4.2

## A.9 Testfall und sein Aufruf auf der Konsole

```
<?php
include(dirname(__FILE__).'/../bootstrap/Propel.php');

$t = new lime_test(13);

$t->comment('Empty Information');
$emptyComparedInformation = new ComparedNaturalModuleInformation(array());
$t->is($emptyComparedInformation->getCatalogSign(), ComparedNaturalModuleInformation::
    EMPTY_SIGN, 'Has no catalog sign');
$t->is($emptyComparedInformation->getSourceSign(), ComparedNaturalModuleInformation::
    SIGN_CREATE, 'Source has to be created');

$t->comment('Perfect Module');
$criteria = new Criteria();
$criteria->add(NaturalmodulenamePeer::NAME, 'SMTAB');
$moduleName = NaturalmodulenamePeer::doSelectOne($criteria);
$t->is($moduleName->getName(), 'SMTAB', 'Right modulename selected');
$comparedInformation = $moduleName->loadNaturalModuleInformation();
$t->is($comparedInformation->getSourceSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Source
    sign shines global');
$t->is($comparedInformation->getCatalogSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Catalog
    sign shines global');
$infos = $comparedInformation->getNaturalModuleInformations();
foreach($infos as $info)
{
  $env = $info->getEnvironmentName();
  $t->is($info->getSourceSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Source sign shines at ' .
      $env);
  if ($env != 'SVNENTW')
  {
    $t->is($info->getCatalogSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Catalog sign shines at
        ' . $info->getEnvironmentName());
  }
  else
  {
    $t->is($info->getCatalogSign(), ComparedNaturalModuleInformation::EMPTY_SIGN, 'Catalog sign is
        empty at ' . $info->getEnvironmentName());
  }
}
?>
```

Listing 1: Testfall in PHP

```
ao-suse-ws1.ao-dom.alte-oldenburger.de - PuTTY
ao-suse-ws1:/srv/www/symfony/natural # ./symfony test:unit ComparedNaturalModuleInformation
1..13
# Empty Information
ok 1 - Has no catalog sign
ok 2 - Source has to be created
# Perfect Module
ok 3 - Right modulename selected
ok 4 - Source sign shines global
ok 5 - Catalog sign shines global
ok 6 - Source sign shines at ENTW
ok 7 - Catalog sign shines at ENTW
ok 8 - Source sign shines at QS
ok 9 - Catalog sign shines at QS
ok 10 - Source sign shines at PROD
ok 11 - Catalog sign shines at PROD
ok 12 - Source sign shines at SVNENTW
ok 13 - Catalog sign is empty at SVNENTW
# Looks like everything went fine.
ao-suse-ws1:/srv/www/symfony/natural #
```

Abbildung 10: Aufruf des Testfalls auf der Konsole

## A.10 Klasse: ComparedNaturalModuleInformation

Kommentare und simple Getter/Setter werden nicht angezeigt.

```php
<?php
class ComparedNaturalModuleInformation
{
  const EMPTY_SIGN = 0;
  const SIGN_OK = 1;
  const SIGN_NEXT_STEP = 2;
  const SIGN_CREATE = 3;
  const SIGN_CREATE_AND_NEXT_STEP = 4;
  const SIGN_ERROR = 5;

  private $naturalModuleInformations = array();

  public static function environments()
  {
    return array("ENTW", "SVNENTW", "QS", "PROD");
  }

  public static function signOrder()
  {
    return array( self :: SIGN_ERROR, self::SIGN_NEXT_STEP, self::
          SIGN_CREATE_AND_NEXT_STEP, self::SIGN_CREATE, self::SIGN_OK);
  }

  public function ___construct(array $naturalInformations)
  {
    $this->allocateModulesToEnvironments($naturalInformations);
    $this->allocateEmptyModulesToMissingEnvironments();
    $this->determineSourceSignsForAllEnvironments();
```

```
28  }
29
30  private function allocateModulesToEnvironments(array $naturalInformations)
31  {
32    foreach ($naturalInformations as $naturalInformation)
33    {
34      $env = $naturalInformation->getEnvironmentName();
35      if (in_array($env, self::environments()))
36      {
37        $this->naturalModuleInformations[array_search($env, self::environments())] = $naturalInformation;
38      }
39    }
40  }
41
42  private function allocateEmptyModulesToMissingEnvironments()
43  {
44    if (array_key_exists(0, $this->naturalModuleInformations))
45    {
46      $this->naturalModuleInformations[0]->setSourceSign(self::SIGN_OK);
47    }
48
49    for ($i = 0;$i < count(self::environments());$i++)
50    {
51      if (!array_key_exists($i, $this->naturalModuleInformations))
52      {
53        $environments = self::environments();
54        $this->naturalModuleInformations[$i] = new EmptyNaturalModuleInformation($environments[$i]);
55        $this->naturalModuleInformations[$i]->setSourceSign(self::SIGN_CREATE);
56      }
57    }
58  }
59
60  public function determineSourceSignsForAllEnvironments()
61  {
62    for ($i = 1; $i < count(self::environments()); $i++)
63    {
64      $currentInformation = $this->naturalModuleInformations[$i];
65      $previousInformation = $this->naturalModuleInformations[$i - 1];
66      if ($currentInformation->getSourceSign() <> self::SIGN_CREATE)
67      {
68        if ($previousInformation->getSourceSign() <> self::SIGN_CREATE)
69        {
70          if ($currentInformation->getHash() <> $previousInformation->getHash())
71          {
72            if ($currentInformation->getSourceDate('YmdHis') > $previousInformation->getSourceDate('
                  YmdHis'))
73            {
74              $currentInformation->setSourceSign(self::SIGN_ERROR);
75            }
76            else
77            {
```

```
78                $currentInformation->setSourceSign(self::SIGN_NEXT_STEP);
79              }
80            }
81            else
82            {
83              $currentInformation->setSourceSign(self::SIGN_OK);
84            }
85          }
86          else
87          {
88            $currentInformation->setSourceSign(self::SIGN_ERROR);
89          }
90        }
91        elseif ($previousInformation->getSourceSign() <> self::SIGN_CREATE && $previousInformation->
            getSourceSign() <> self::SIGN_CREATE_AND_NEXT_STEP)
92        {
93          $currentInformation->setSourceSign(self::SIGN_CREATE_AND_NEXT_STEP);
94        }
95      }
96    }
97
98    private function containsSourceSign($sign)
99    {
100     foreach($this->naturalModuleInformations as $information)
101     {
102       if($information->getSourceSign() == $sign)
103       {
104         return true;
105       }
106     }
107     return false;
108   }
109
110   private function containsCatalogSign($sign)
111   {
112     foreach($this->naturalModuleInformations as $information)
113     {
114       if($information->getCatalogSign() == $sign)
115       {
116         return true;
117       }
118     }
119     return false;
120   }
121 }
122 ?>
```

Listing 2: Klasse: ComparedNaturalModuleInformation

## A.11 Klassendiagramm

Klassendiagramme und weitere UML-Diagramme kann man auch direkt mit LaTeX zeichnen, siehe z. B. http://metauml.sourceforge.net/old/class-diagram.html.

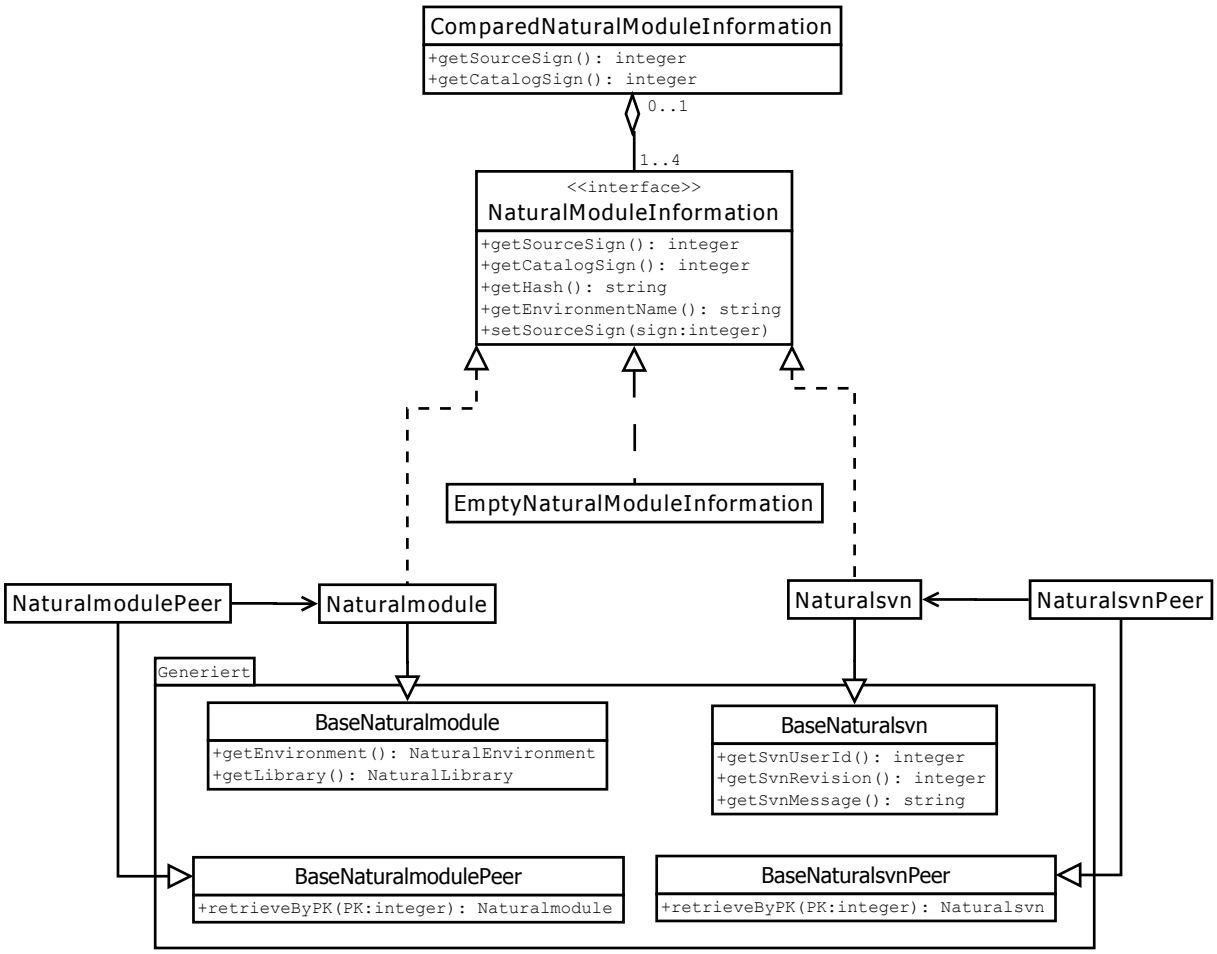


Abbildung 11: Klassendiagramm

## A.12 Benutzerdokumentation

Ausschnitt aus der Benutzerdokumentation:

| Symbol | Bedeutung global | Bedeutung einzeln |
|---|---|---|
| | Alle Module weisen den gleichen Stand auf. | Das Modul ist auf dem gleichen Stand wie das Modul auf der vorherigen Umgebung. |
| | Es existieren keine Module (fachlich nicht möglich). | Weder auf der aktuellen noch auf der vorherigen Umgebung sind Module angelegt. Es kann also auch nichts übertragen werden. |
| | Ein Modul muss durch das Übertragen von der vorherigen Umgebung erstellt werden. | Das Modul der vorherigen Umgebung kann übertragen werden, auf dieser Umgebung ist noch kein Modul vorhanden. |
| | Auf einer vorherigen Umgebung gibt es ein Modul, welches übertragen werden kann, um das nächste zu aktualisieren. | Das Modul der vorherigen Umgebung kann übertragen werden um dieses zu aktualisieren. |
| | Ein Modul auf einer Umgebung wurde entgegen des Entwicklungsprozesses gespeichert. | Das aktuelle Modul ist neuer als das Modul auf der vorherigen Umgebung oder die vorherige Umgebung wurde übersprungen. |